Lecture 9: Training a binary classification model

BINF 4002

Feedback!

Form Link (will be distributed via courseworks):
https://forms.gle/LiXN5CuLf7qZmBvT8

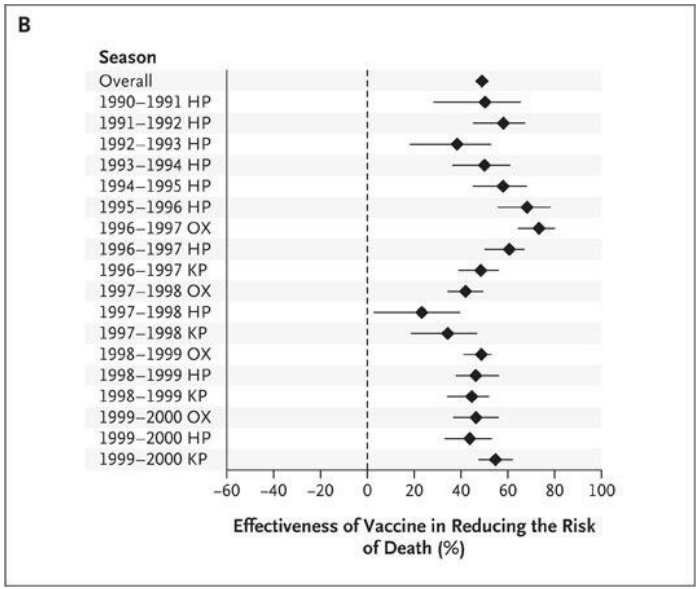
Coming Up

- More ML Training!
- ML Losses
- ML Data Types
- ML Models
- ML Algorithms
- Spring Break!
- Health/bio data modalities
- Health/bio specific ML techniques

Influenza Vaccine Efficacy in the Elderly

Nichol et al report a reduction in all-cause mortality of ~50% in the elderly due to the influenza vaccine.

Is there a problem?



Nichol, K. L., Nordin, J. D., Nelson, D. B., Mullooly, J. P., & Hak, E. (2007). Effectiveness of influenza vaccine in the community-dwelling elderly. New England Journal of Medicine, 357(14), 1373-1381.

Outline

1. Last time: Evaluation of a binary classification model.
2. This time: How do we make that f in the first place?
   (at a high level)

At a high level, the ML pipeline is:
Identify loss, Optimize, Don’t overfit.

- Loss is any function that approximates your deployment/use-case error.
- Optimize some class of learnable functions to minimize that loss on an observed sample of data.
- Don’t overfit; leverage things like hyperparameter tuning, monitoring, regularization.

Outline

1. Last time: Evaluation of a binary classification model.
    a. Given a *fixed* model f, how do we judge performance?
2. This time: How do we make that f in the first place?
    a. Before the model:
        i. Data Splits
        ii. Model capacity / misspecification
    b. Procedural View
        i. K-nearest-neighbors
            1. Consistent Learning Framework
        ii. Optimization -- Performance is Efficiency
    c. Optimization View
        i. Solve a loss function
        ii. What is our goal? Perfect or not perfect loss?

Notation and Setup

Let our model be a function $f_\vartheta : \boldsymbol{x} \mapsto s \in \mathbb{R}$ trained on a binary classification task over a dataset $\boldsymbol{X} \times \mathbf{y}$, $y_i \in \{0, 1\}$, such that for input $\boldsymbol{x}$, our model produces *score* $\boldsymbol{s}$.

Let us define the random variables $\mathbf{x}$, $\mathbf{y}$, $\boldsymbol{s}$ to correspond to the inputs, labels, and model produced scores---so $\boldsymbol{s} = f_\vartheta(\mathbf{x})$---with probability density functions $p_{\mathbf{x}}, p_{\mathbf{y}}, p_{\boldsymbol{s}}$.

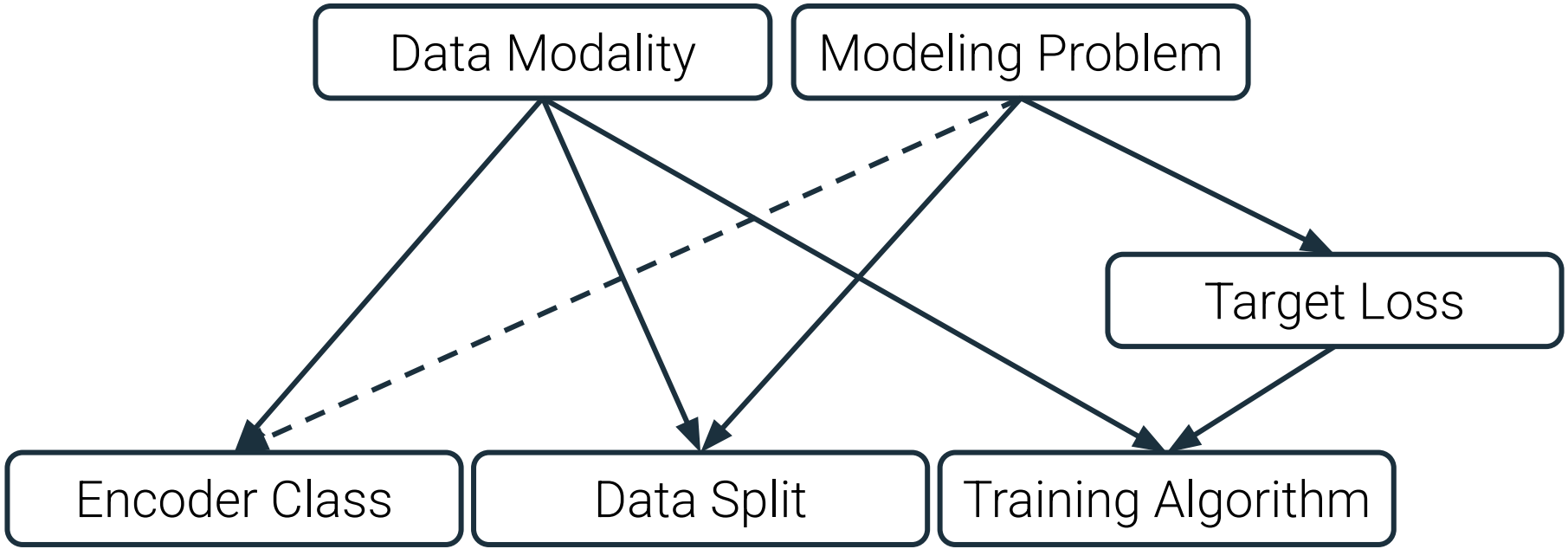
Let us further define the random variables $\boldsymbol{s}_1$ and $\boldsymbol{s}_0$ to correspond to the scores returned only when the true labels are 1 and 0, respectively, with probability density functions q_1 and q_0. At times we may assume that $\boldsymbol{s}$ is normalized to a probability output.

Let's define $R : s \mapsto \{0, 1\}$ be our decision rule, so we predict label $\hat{y} = R(f_\vartheta(\boldsymbol{x}))$.

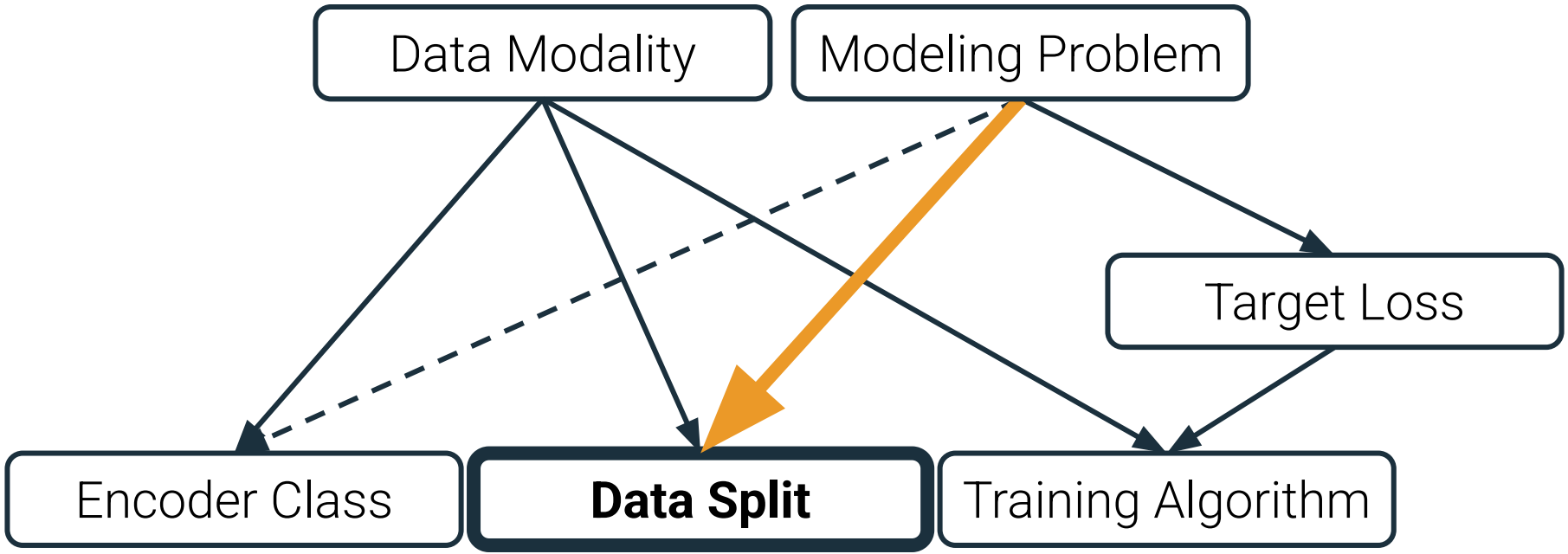
What do we do first?

We need to know what we’re trying to model!

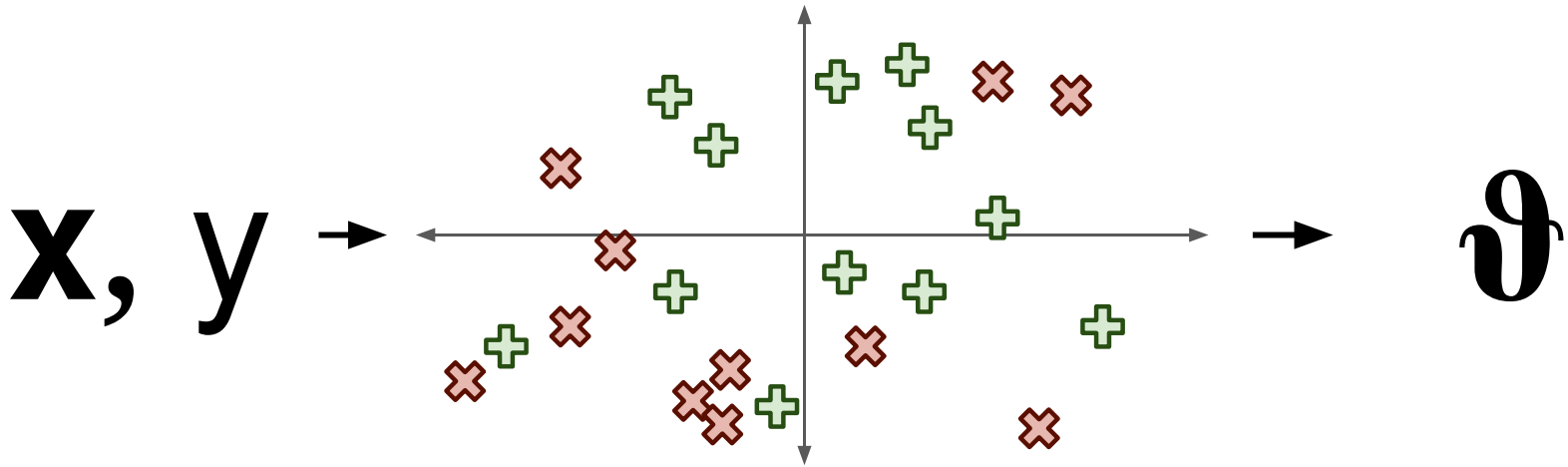
The Problem (and data) Determine All



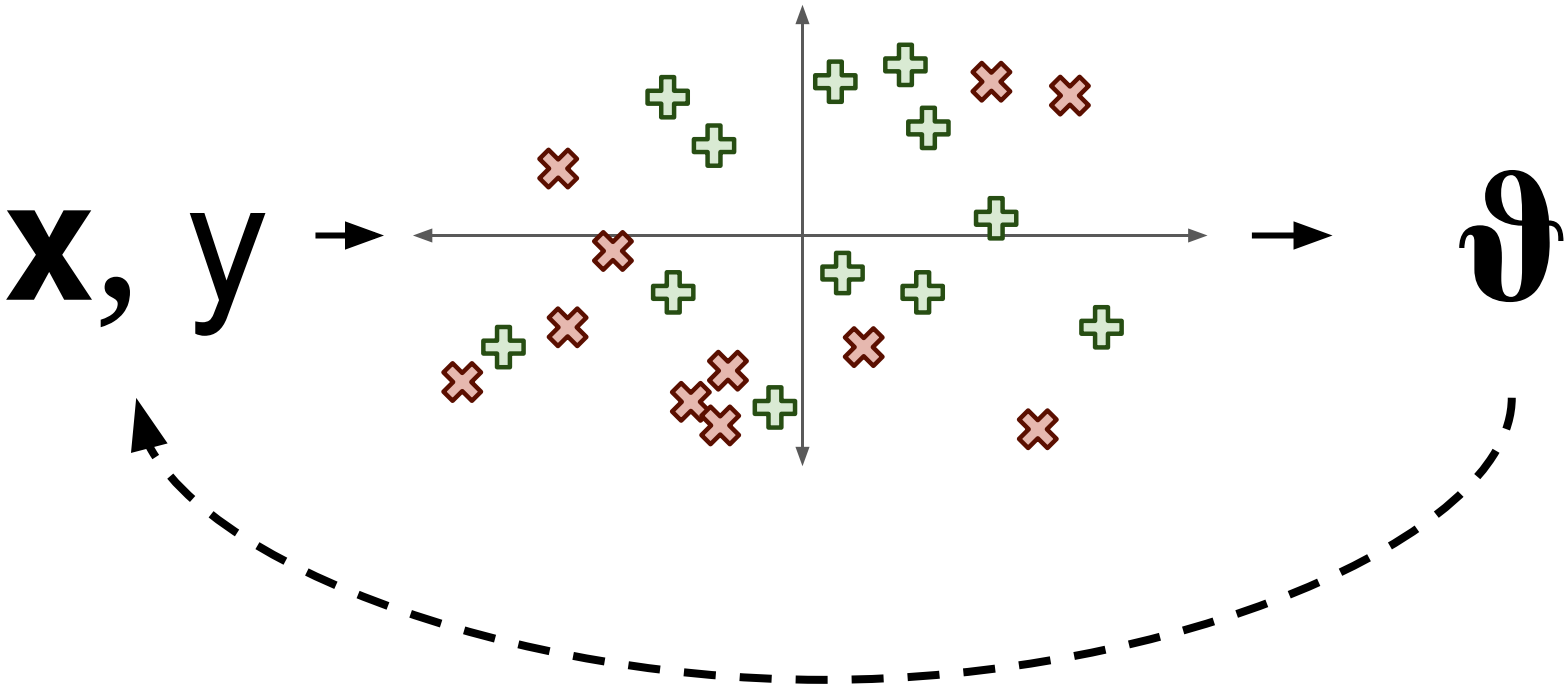
The Problem (and data) Determine All



The Information Pipeline in ML

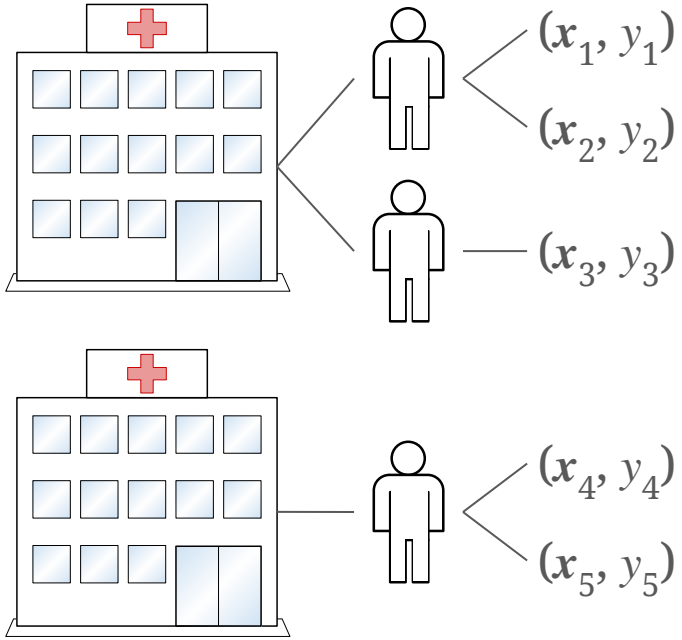


The Information Pipeline in ML

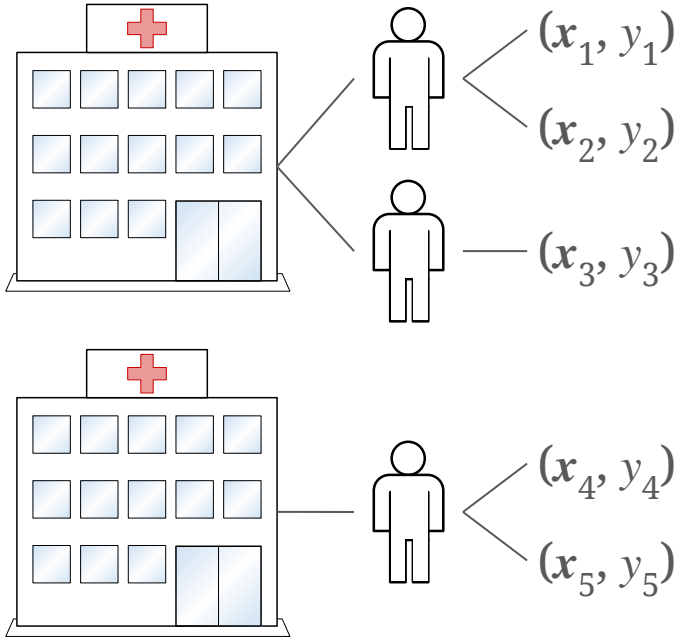


Splitting your data lets you assess performance on the underlying data distribution-not your sampled dataset!

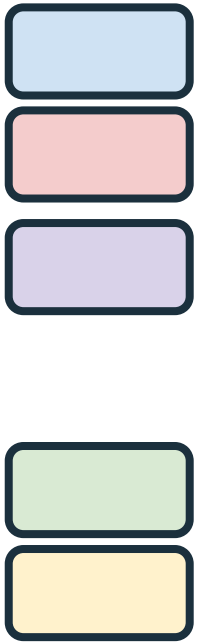
Target Use-case Implies Target Generalizability Unit



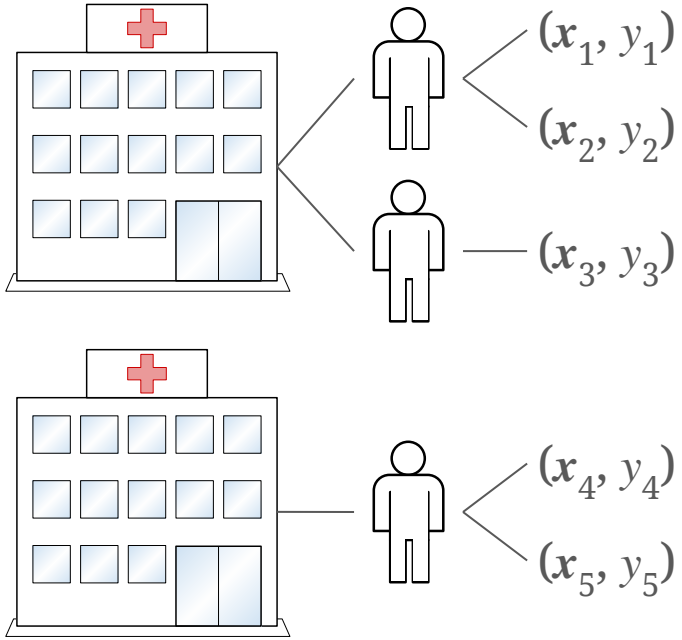
Target Use-case Implies Target Generalizability Unit

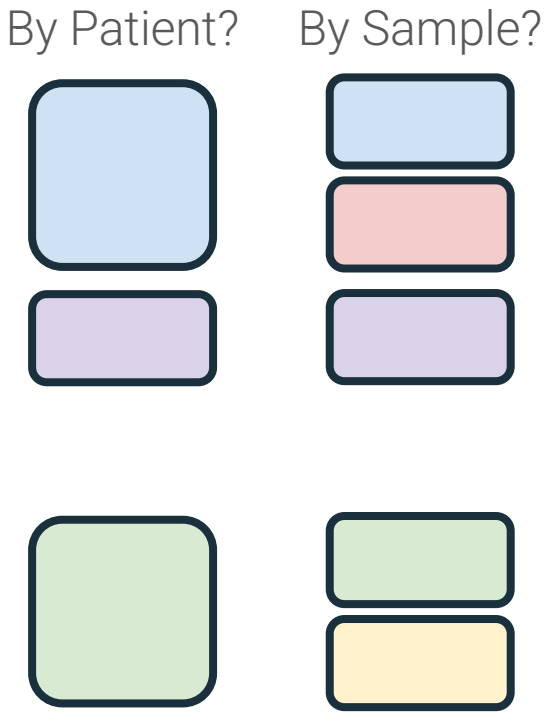


By Sample?

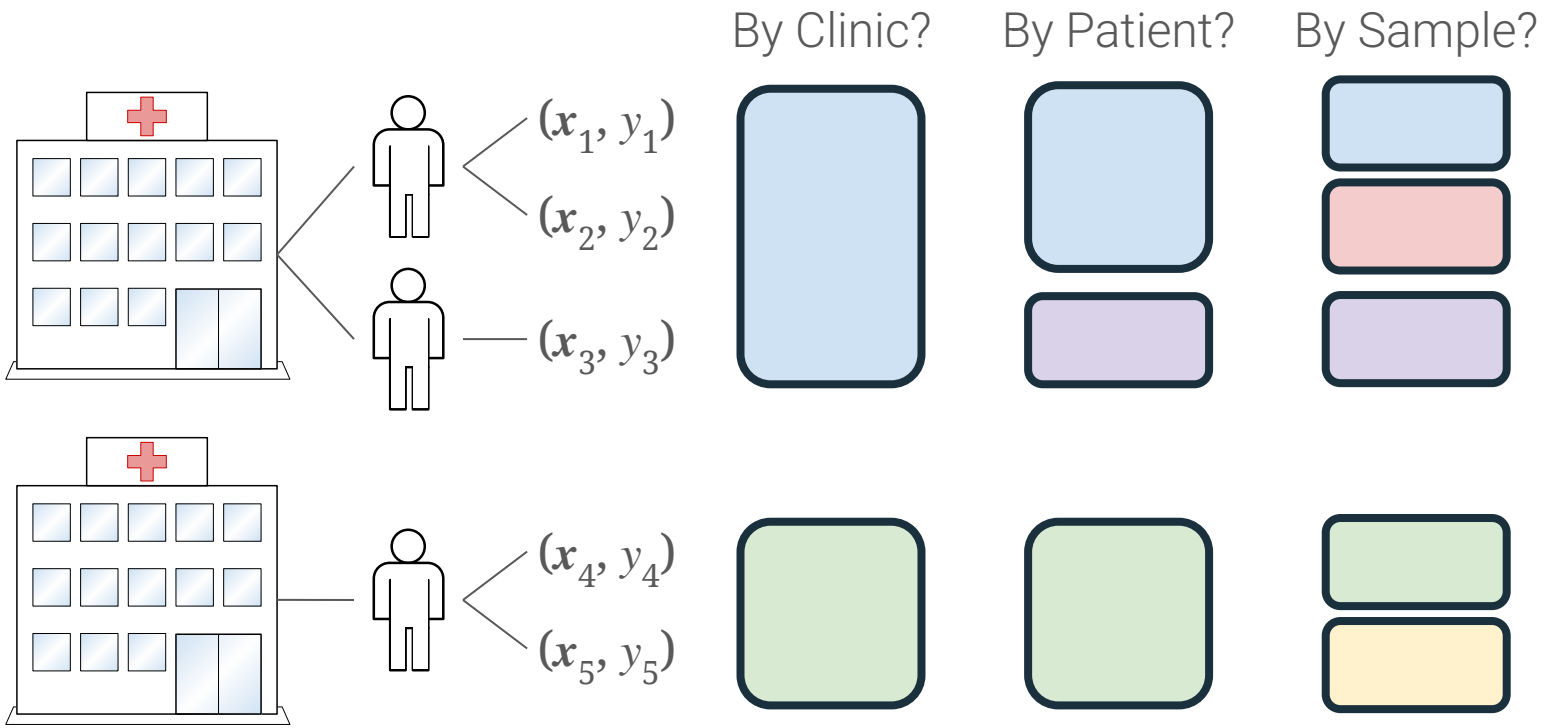


Target Use-case Implies Target Generalizability Unit

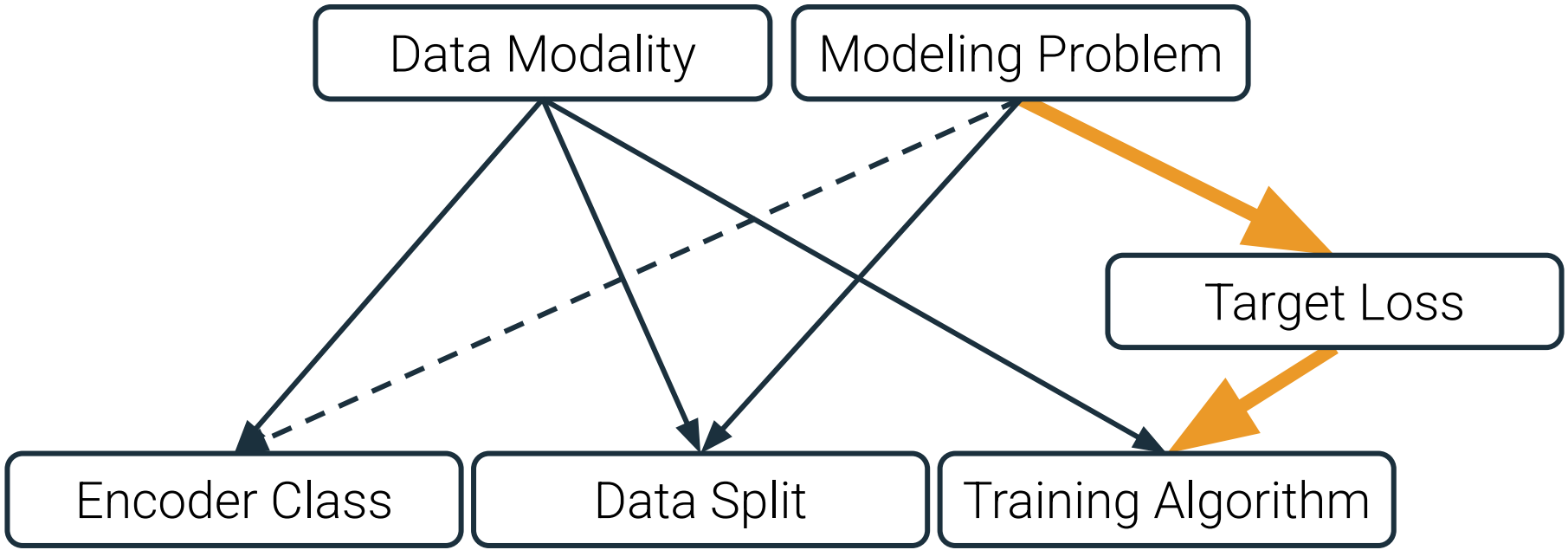




Target Use-case Implies Target Generalizability Unit



The Problem (and data) Determine All

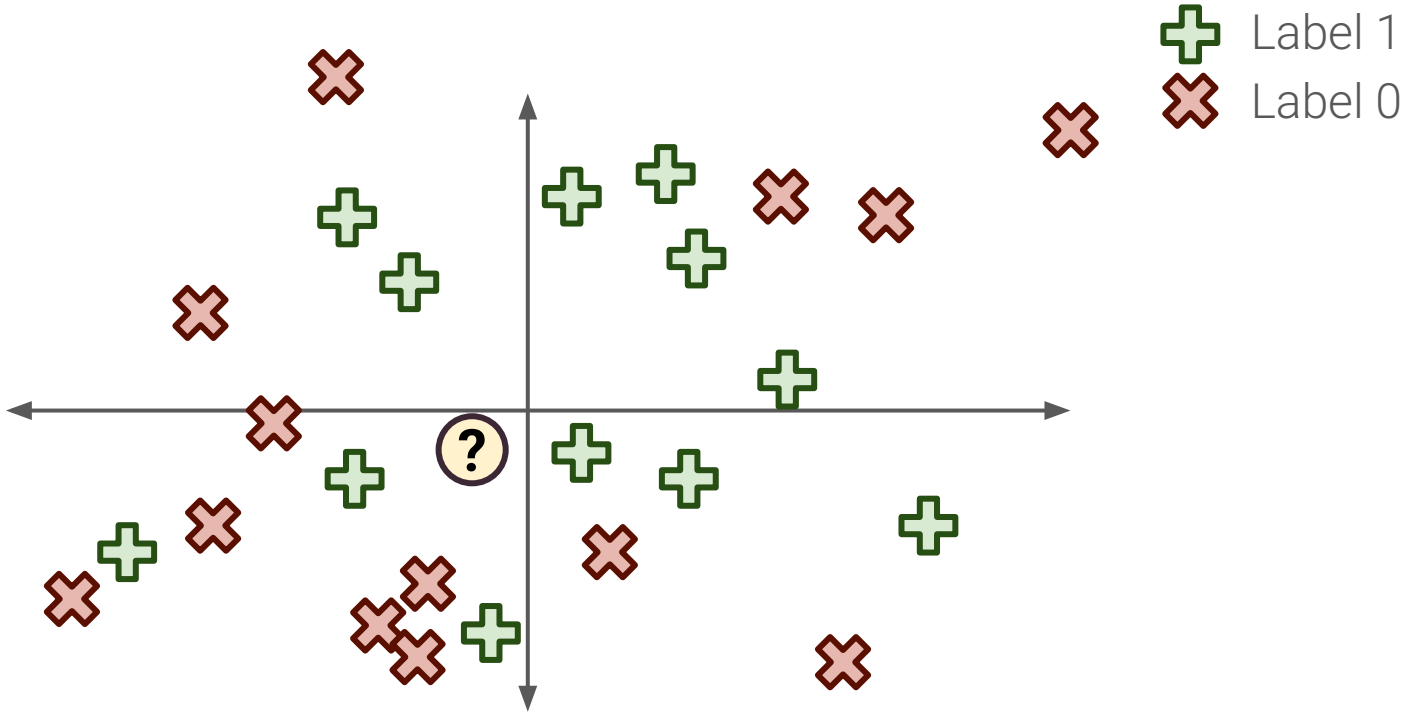


A common loss variety

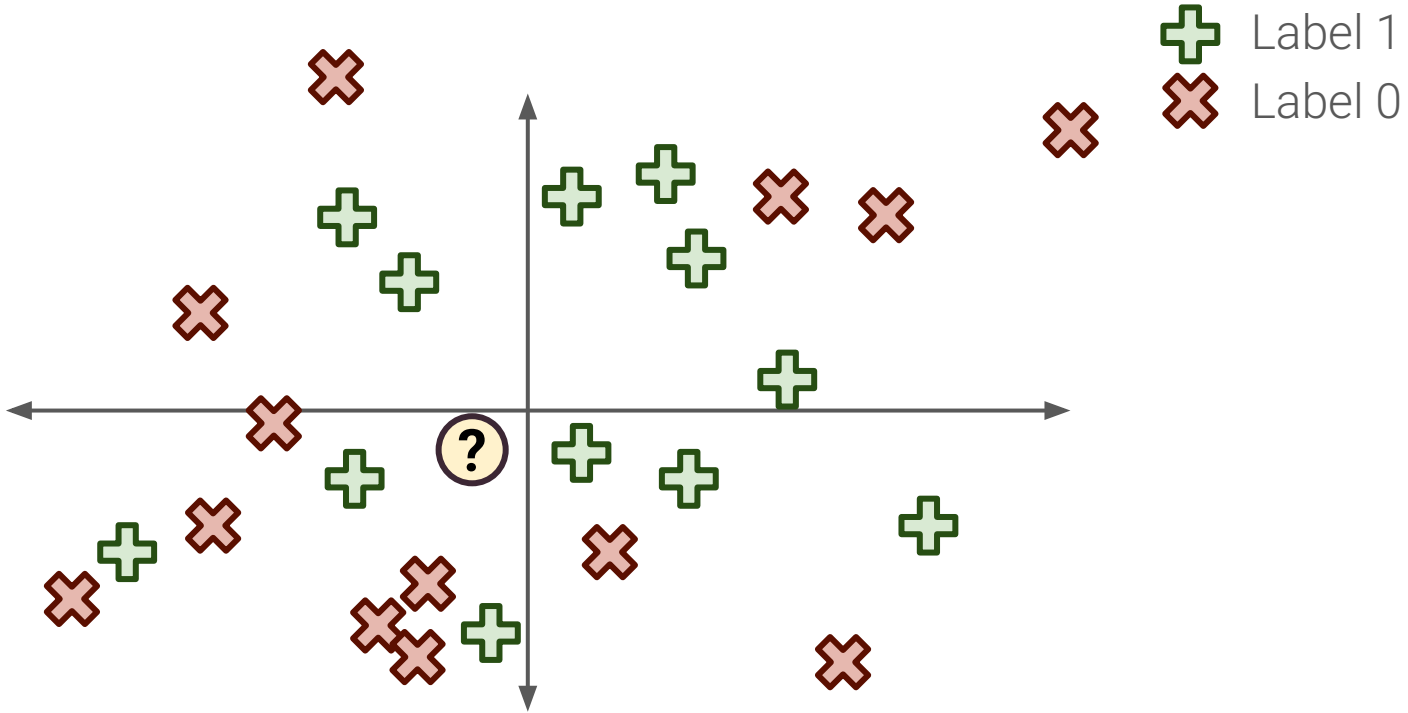
- Define a generative model of your data random variables **x**, **y**.
- You can model both **x**, **y** or just **y** | **x**.
- Identify the parameters for that generative model that maximize the likelihood of the data you observe.
- This results in minimizing a “negative log likelihood” loss.

The question, then?
How do we model
$\mathbf{x}$, y or $y \mid \mathbf{x}$.

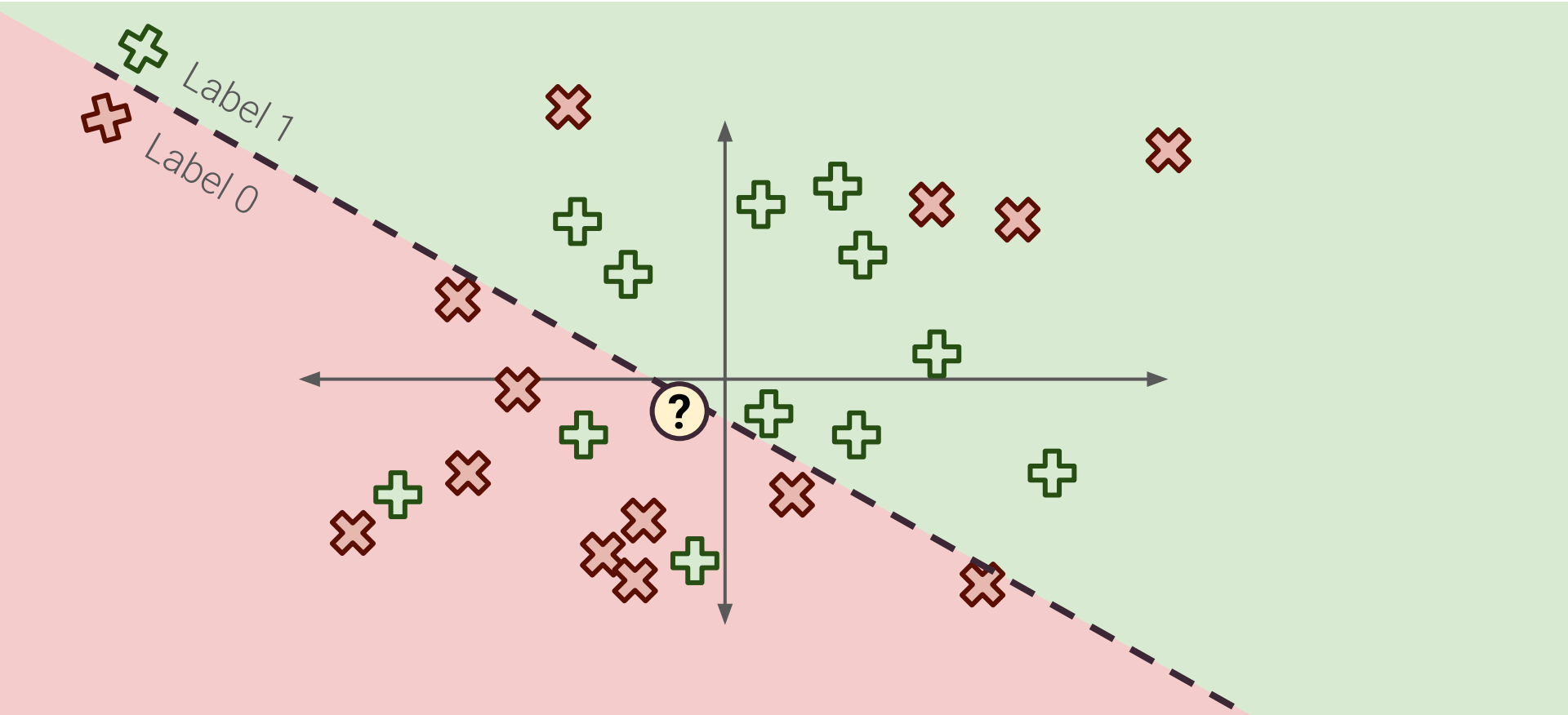
What is the simplest model we can use?



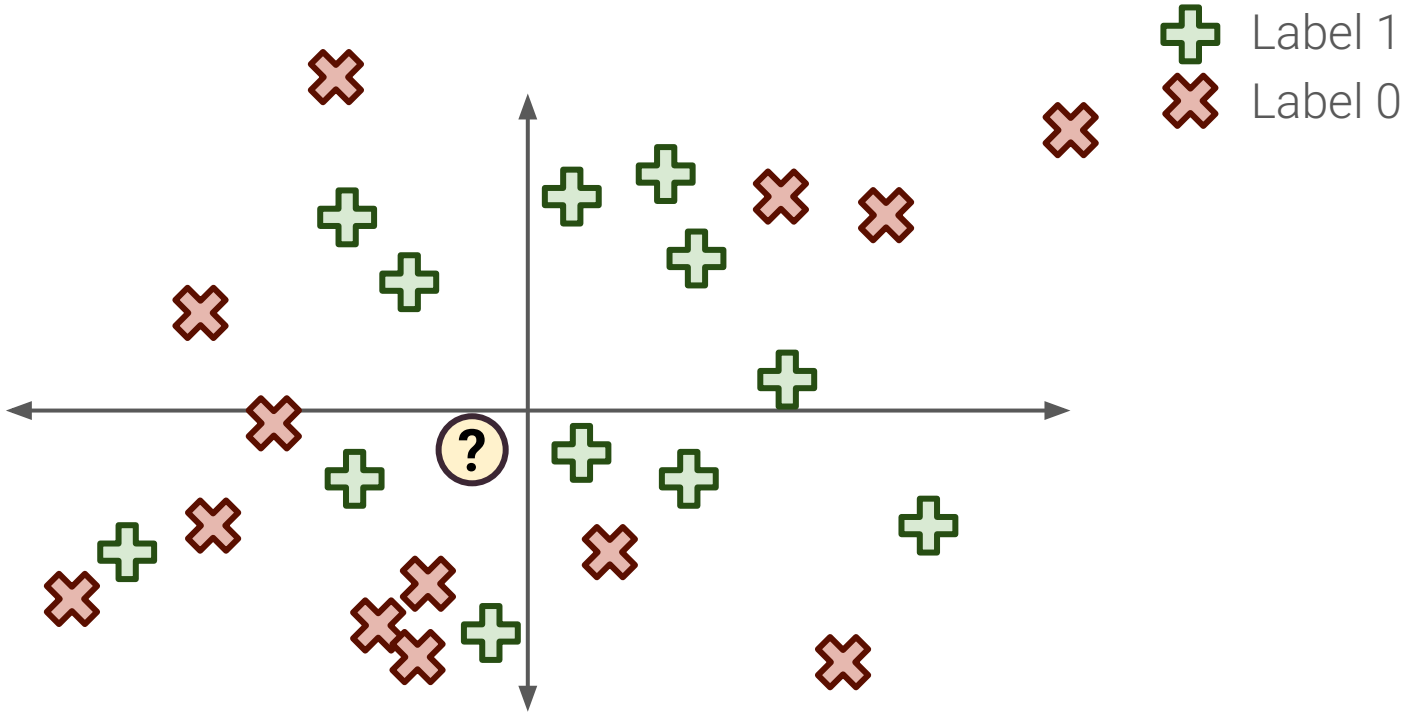
What is the simplest *non-trivial* model we can use?



Let’s separate the classes linearly?

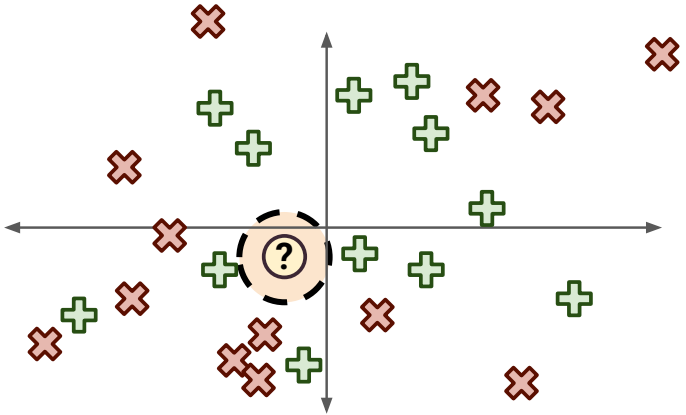


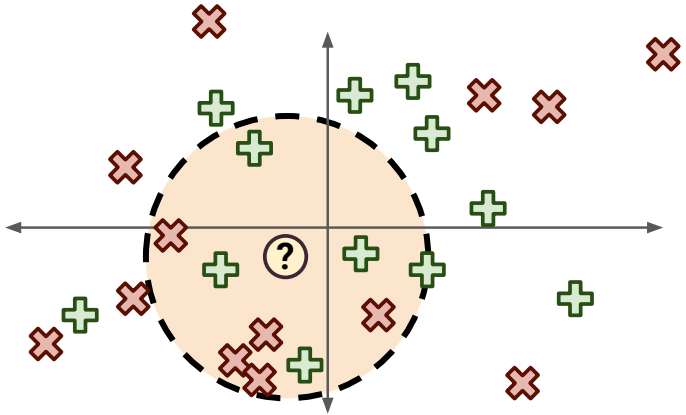
Let’s guess what we’ve seen before!



What are we learning in the nearest neighbor model?

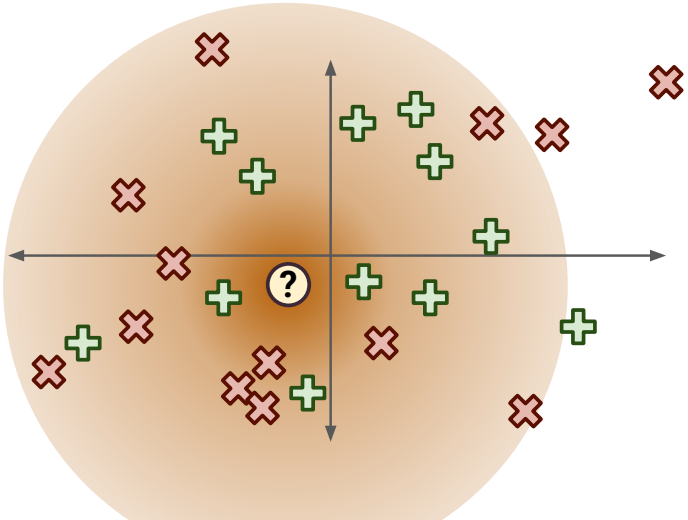
- *k* (number of neighbors) or *r* (radius of neighborhood)

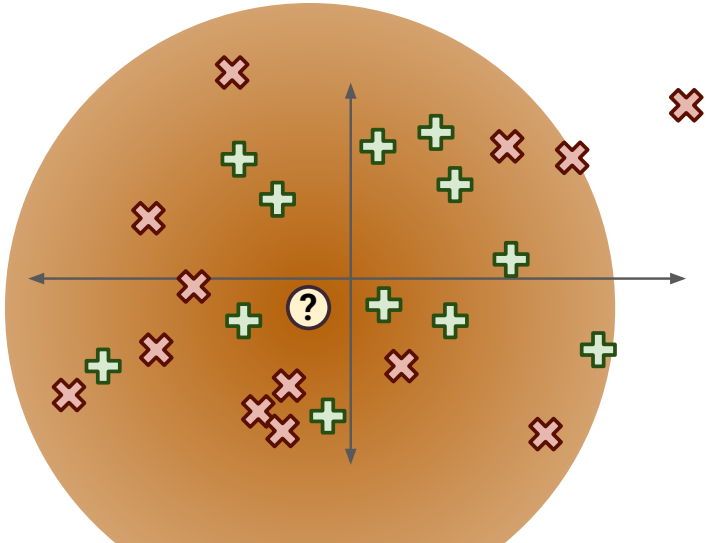




What are we learning in the nearest neighbor model?

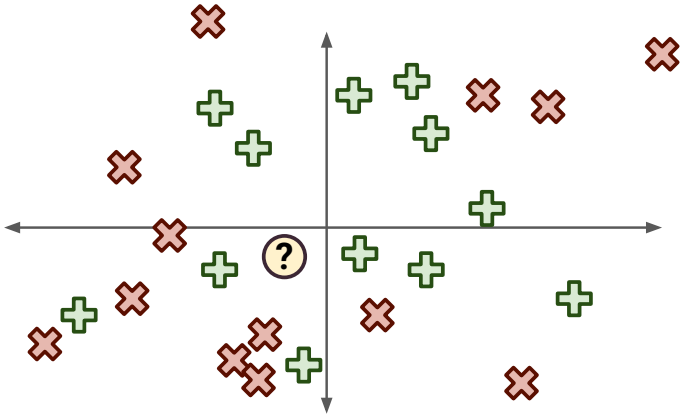
- *k* (number of neighbors) or *r* (radius of neighborhood)?
- Kernel function (weighting distant vs. close neighbors)?

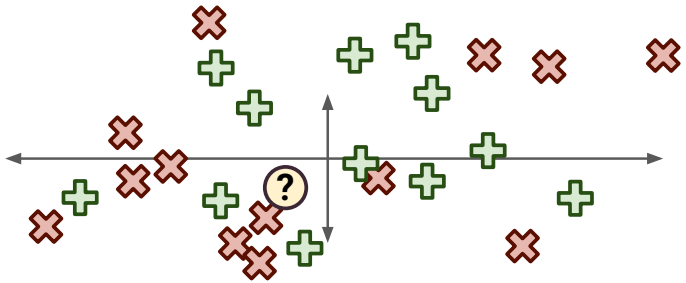




What are we learning in the nearest neighbor model?

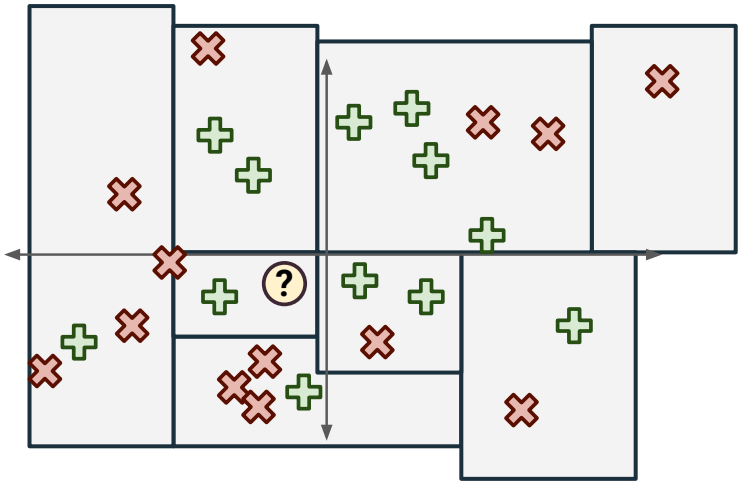
- *k* (number of neighbors) or *r* (radius of neighborhood)?
- Kernel function (weighting distant vs. close neighbors)?
- Distance function itself!





What are we learning in the nearest neighbor model?

- *k* (number of neighbors) or *r* (radius of neighborhood)?
- Kernel function (weighting distant vs. close neighbors)?
- Distance function itself!
- Index to make inference efficient?



How do we train these?

Hyperparameters vs. Parameters?

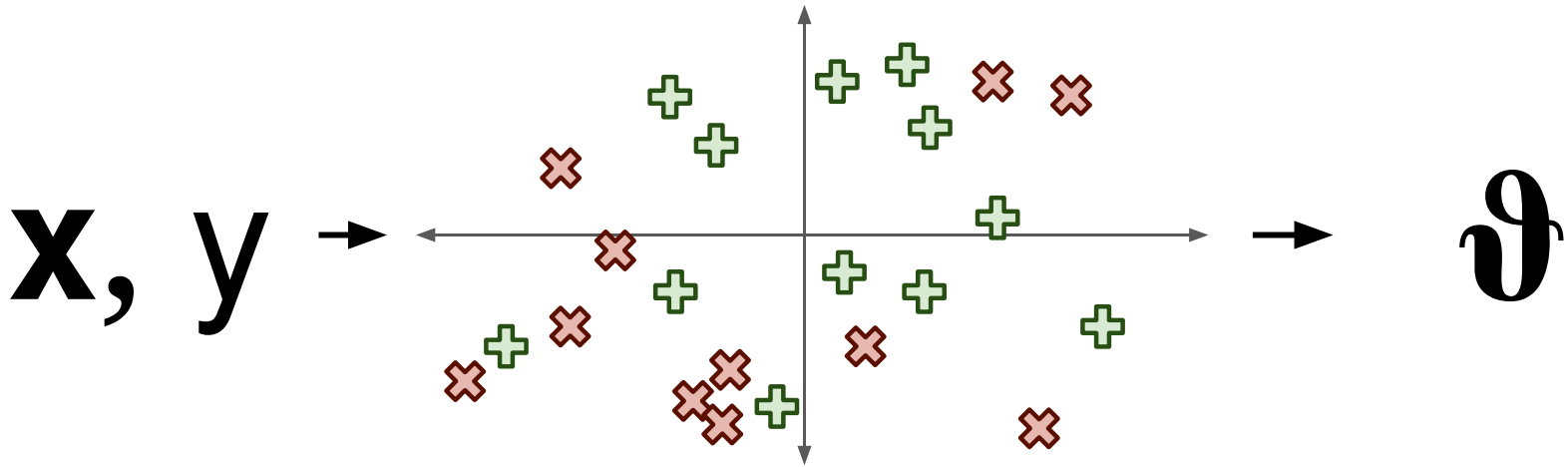
Hyperparameters

- Usually Non-differentiable
- May only indirectly influence the per-sample level prediction/loss
- Often fit on the “validation” / “dev” / “tuning” set.

Parameters

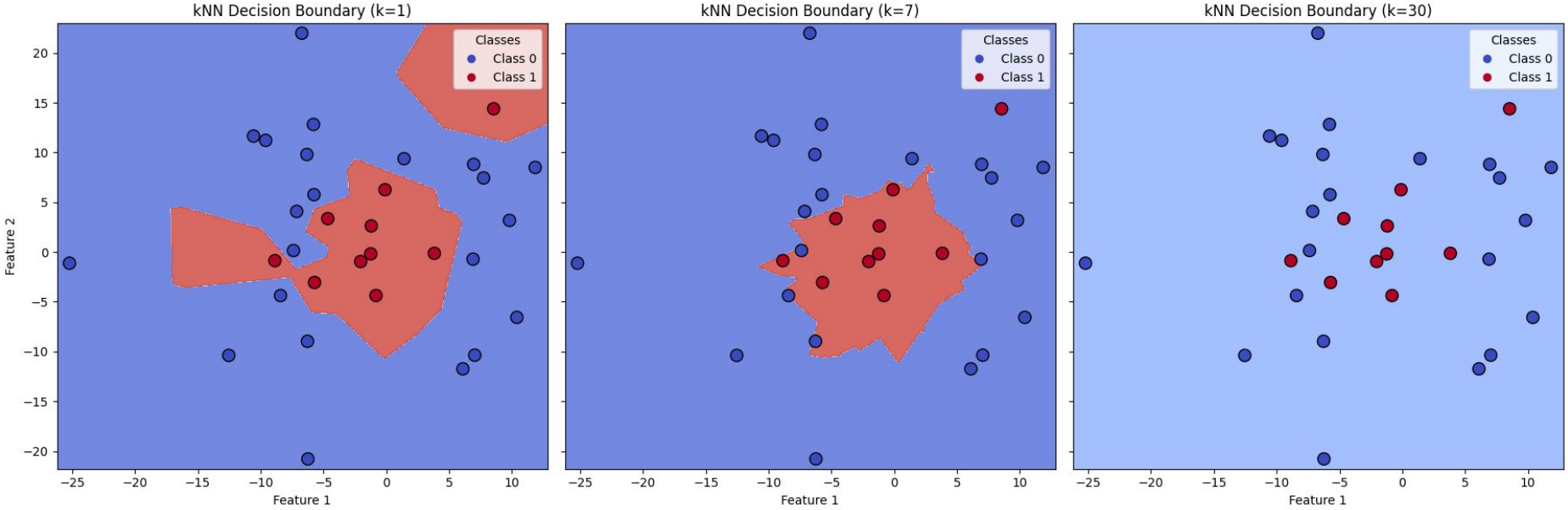
- Usually Differentiable
- Usually directly influence the per-sample level prediction/loss.
- Fit on the “train” set

What information is being transferred?

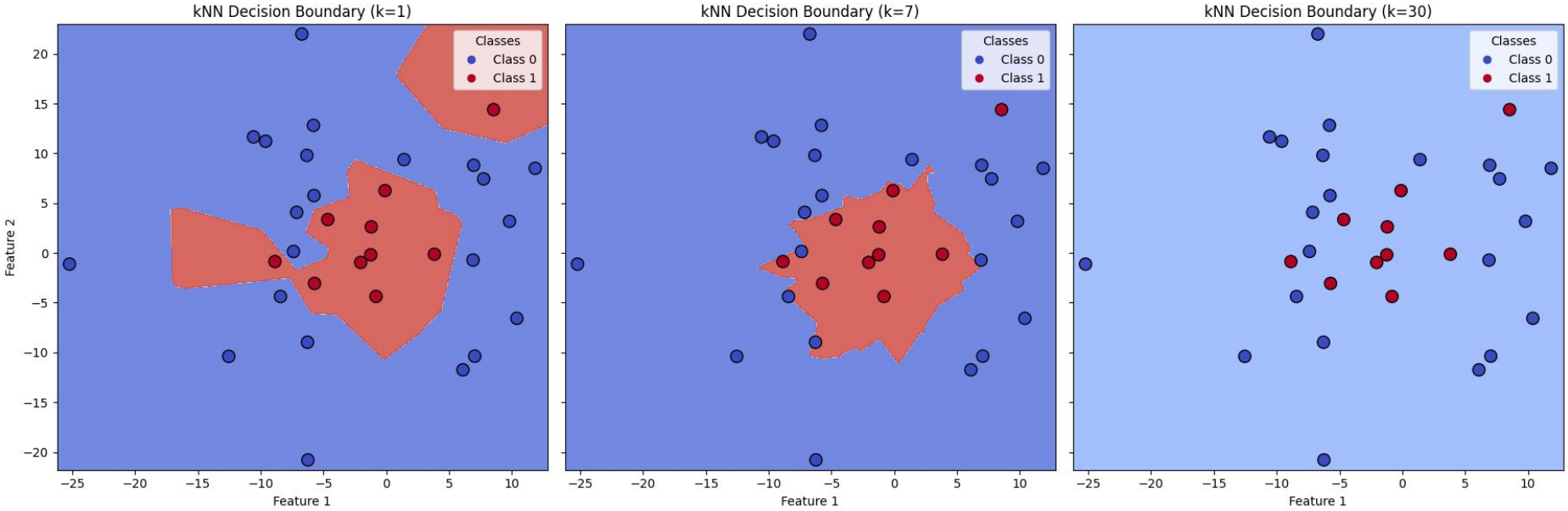


Expected Error:
Bias and Variance

How do these values affect our expected error?



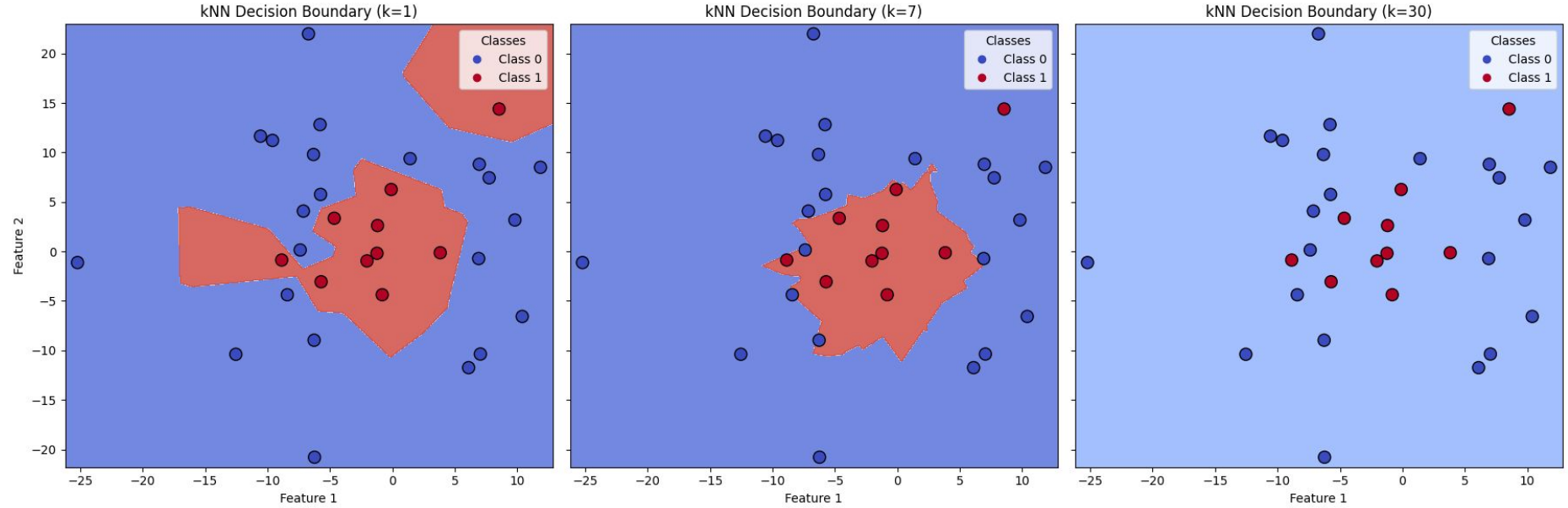
How do these values affect our expected error?

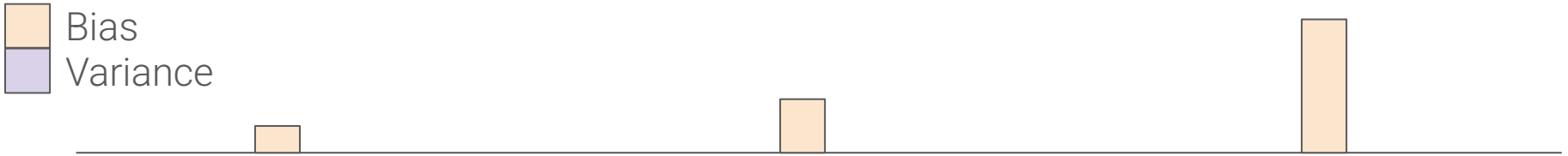


Bias

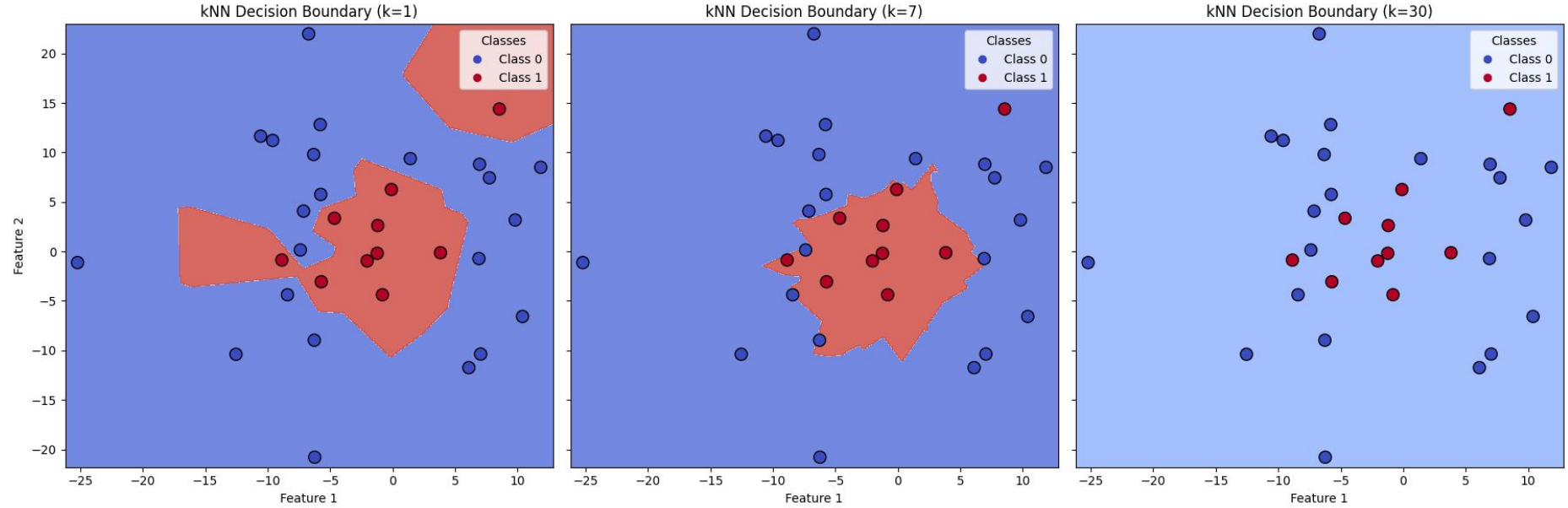
Variance

How do these values affect our expected error?





How do these values affect our expected error?



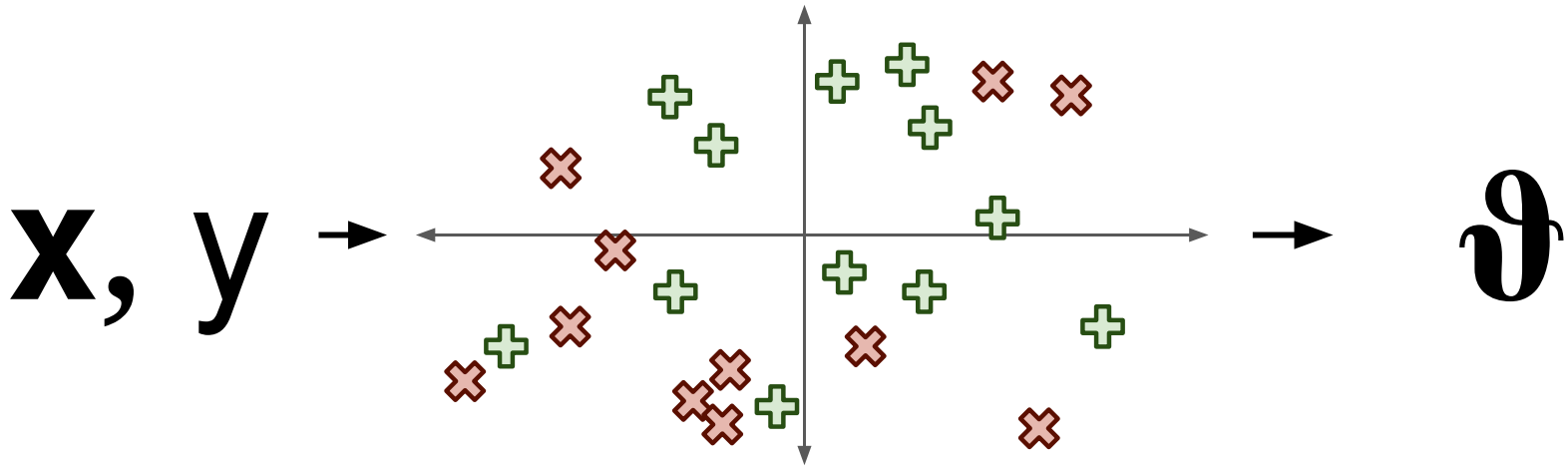
What are “bias” and “variance”?

Informally / Quasi-formally

- An estimator is *unbiased* if, over the statistical process used to estimate it and all sources of randomness, its expected value equals the target property. If not, it is (statistically) *biased*.
- The “variance” of an estimator refers to how much deviation any given sample of this estimate will take from its expected value.
- The “bias” in the bias-variance tradeoff thus refers to whether a model will actually model the “right thing” and the variance refers to how much the randomness in the learning process will add uncertainty to our output.

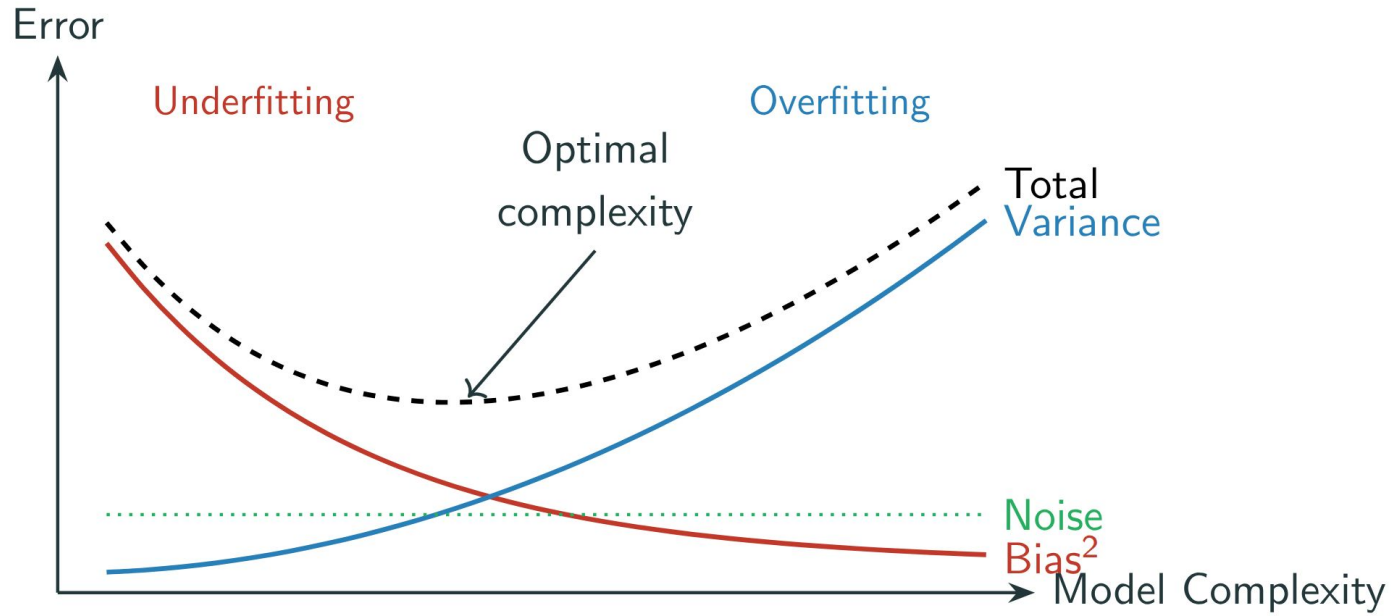
What is the “right thing”?

We are learning over a (binary classification) problem with random variables **x**, **y**



The “right” output for our model to learn is precisely $g(\boldsymbol{x}) = p_{\mathrm{y} \mid \mathbf{x}=\boldsymbol{x}}$

Model “Capacity” or “Complexity” reflects the model’s ability to fit complex functions. As capacity increases, bias decreases and variance increases.

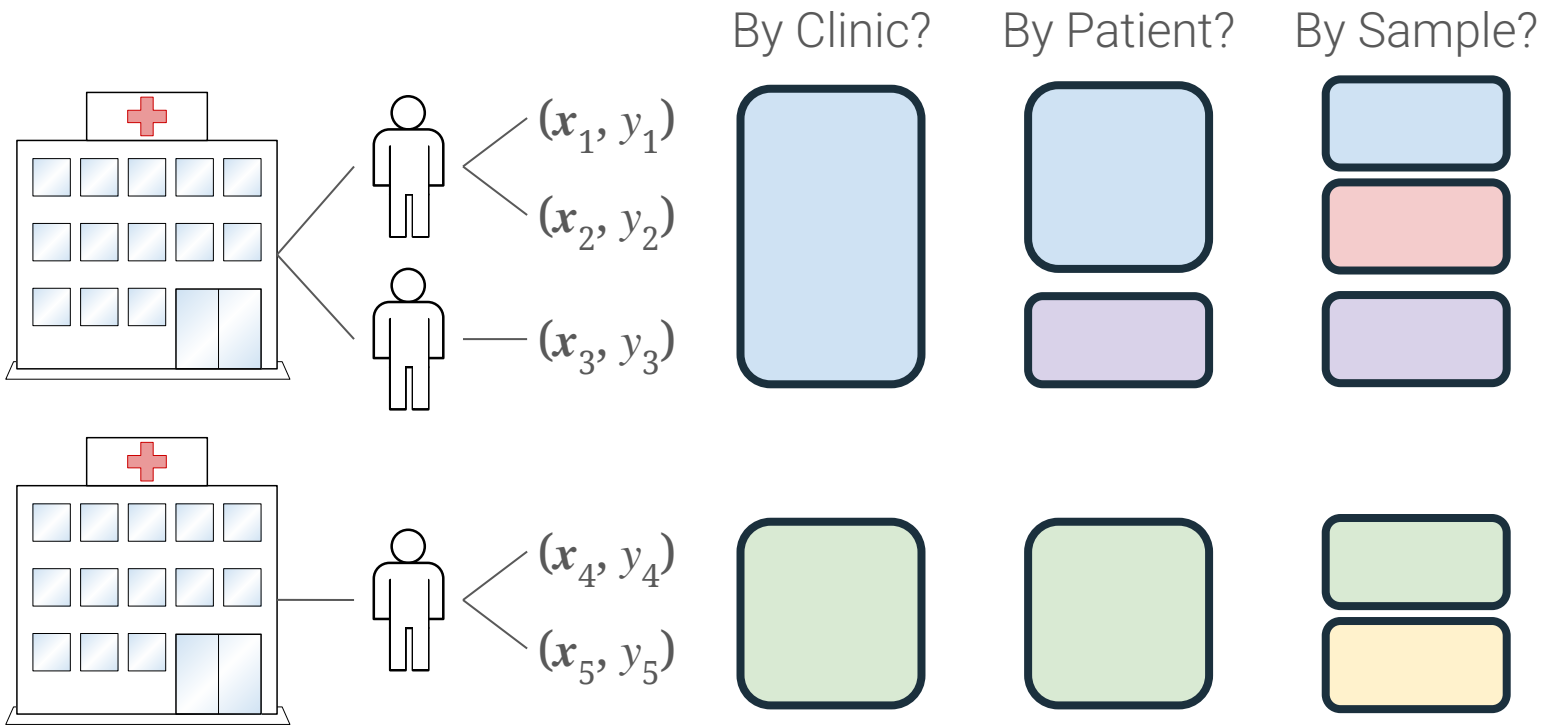


What are the sources of this “variance” / “uncertainty”?

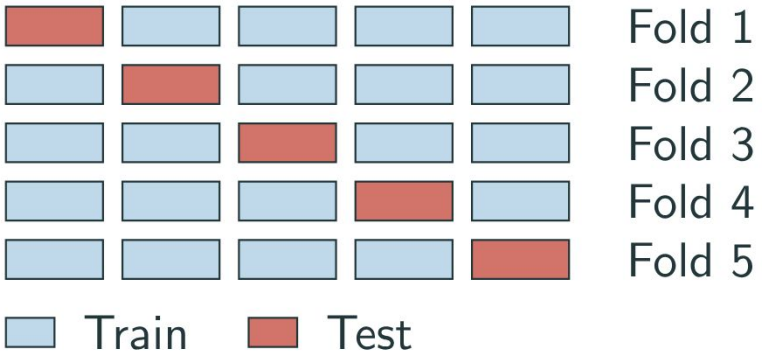
1. Training dataset sample.
2. Hyperparameter choice.
3. Parameter Initialization.
4. Stochasticity in algorithm.
5. Test sample*.

How do we assess uncertainty?

We may need to repeat our split many times



We may need to repeat our split many times



Expected error draws from three sources

Bias-Variance Decomposition

$$\mathbb{E}\left[\left(Y - \hat{f}(x)\right)^2\right] = \underbrace{\sigma^2}_{\text{Noise}} + \underbrace{\left(\mathbb{E}[\hat{f}(x)] - f(x)\right)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right]}_{\text{Variance}}$$

- **Noise** (σ^2): Irreducible — even the perfect model can't predict random variation
- **Bias**^{2}: How far is our *average* prediction from the truth?
- **Variance**: How much does $\hat{f}(x)$ change across different training sets?

WARNING: Not mathematically same objects as MSE.

Bias-Variance Decomposition (0-1 Loss)

$$\mathbb{E}\left[\mathbf{1}_{Y \neq \hat{y}}\right] = \underbrace{p(1-p)}_{\text{Noise}} + \underbrace{(q-p)^2}_{\text{Bias}^2} + \underbrace{q(1-q)}_{\text{Variance}}$$

- **Noise**: Bayes error — unavoidable when $p \neq 0, 1$
- **Bias^2**: Systematic misestimation of $P(Y = 1)$
- **Variance**: Instability in predicted class across training sets

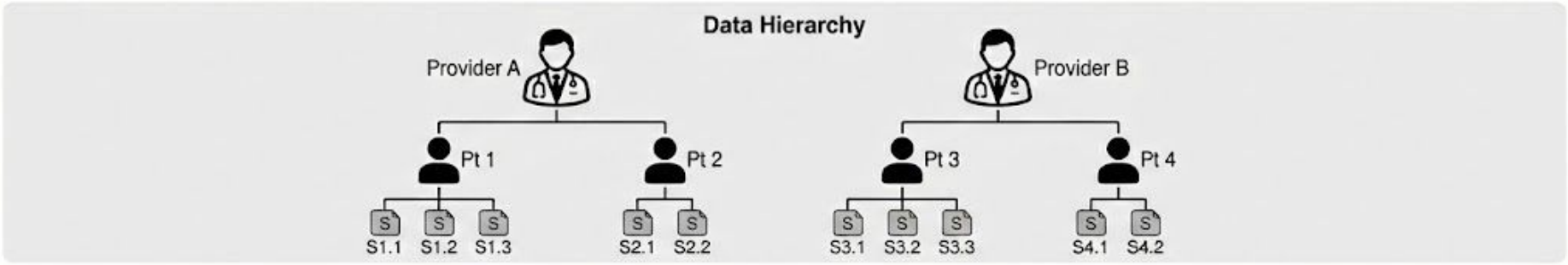
Most General

$$R(f) := \mathbb{E}[\ell(Y, f(X))] \quad \text{and} \quad f^* := \arg\min_f R(f)$$

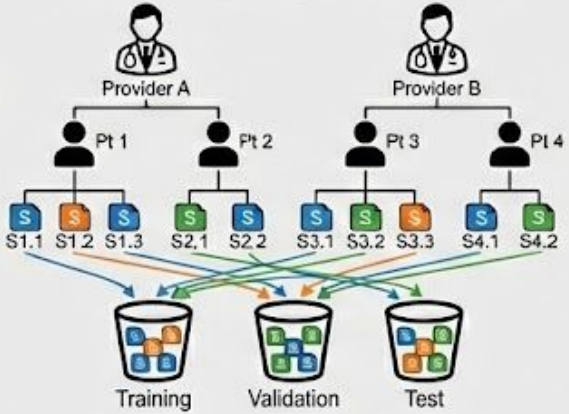
$$\mathbb{E}_D[R(\hat{f}_D)] - R(f^*) = \underbrace{\left(R(f^*_{\mathcal{F}}) - R(f^*)\right)}_{\text{approximation error (“bias”)}} + \underbrace{\mathbb{E}_D\left[R(\hat{f}_D) - R(f^*_{\mathcal{F}})\right]}_{\text{estimation error (“variance”)}}.$$

Healthcare ML Data Splits: Respecting Structure (Provider -> Patient -> Sample)

Training Set (Blue) Validation Set (Orange) Test Set (Green)

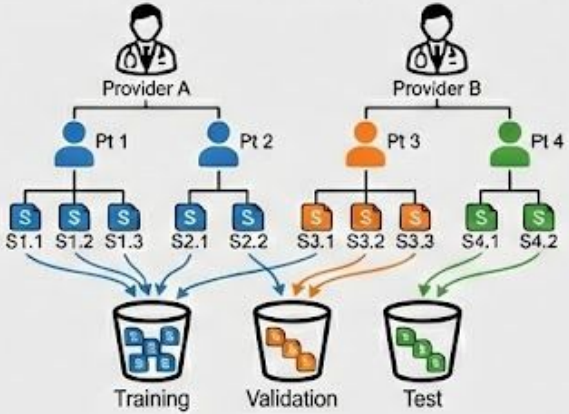


1. Random Split (Ignores Structure)



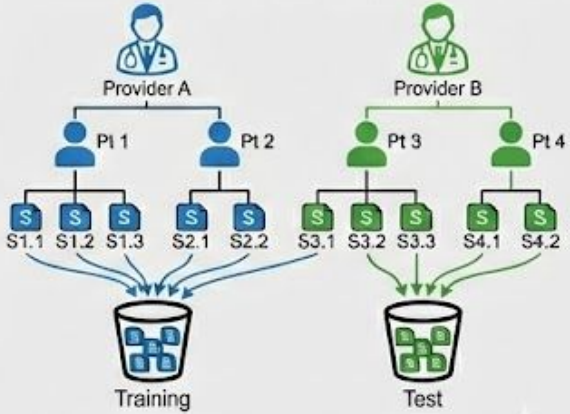
Samples from the same patient or provider can be in different splits. High risk of data leakage.

2. Patient-Level Split (Respects Patient)



All samples from a single patient are in one split. Prevents patient-level leakage.

3. Provider-Level Split (Respects Provider)



All samples from a single provider are in one split. Tests generalizability to new providers.